WEB SYSTEMS DESIGN – PROJECT 5

VISTULA

UNIVERSITY

Name: Amjad Massaoud

Student ID: 78706

Course: Web Systems Design

Semester: 6th semester 57DPH

Date: 30.08.2026

Step 1 - Verify system baseline

I verified that the Docker Compose environment was running correctly. The backend, database, Redis, and Nginx services were checked using docker compose ps. The health endpoint was also tested to confirm that the API was available.

```
jad@fedora-2:~/uni-project/project5/backend$ docker compose ps
NAME                  IMAGE                  SERVICE         STATUS
                PORTS
wsd-project-backend-1  wsd-project-backend      backend         Up 17 hours (healthy)
   0.0.0.0:8000->8000/tcp
wsd-project-db-1       postgres:16-alpine       db              Up 17 hours (healthy)
   5432/tcp
wsd-project-redis-1    redis:7-alpine           redis           Up 17 hours (healthy)
   6379/tcp
wsd-project-web-1      wsd-project-web          web             Up 17 hours (healthy)
   0.0.0.0:8080->80/tcp
```

http://127.0.0.1:8080/api/tasks

```
[ ]
```

http://127.0.0.1:8080/api/health

```
{"status": "ok"}
```

Step 2 - Introduce API versioning

In this step, I updated the Laravel API routes by adding a versioned route prefix. The task API is now available under **/api/78706/v1/tasks**. This structure makes the API easier to maintain and allows future versions, such as /v2, to be added without breaking existing clients.

```
jad@fedora-2:~/uni-project/project5/backend$ php artisan route:list --path=api
GET|HEAD   api/78706/v1/tasks ..............................  Api\TaskController@index
POST       api/78706/v1/tasks ..............................  Api\TaskController@store
GET|HEAD   api/78706/v1/tasks/{id} .........................  Api\TaskController@show
PUT        api/78706/v1/tasks/{id} .........................  Api\TaskController@update
DELETE     api/78706/v1/tasks/{id} .........................  Api\TaskController@destroy
GET|HEAD   api/health ..................................................

Showing [6] routes
```

```
jad@fedora-2:~/uni-project/project5$ curl http://127.0.0.1:8080/api/78706/v1/tasks
[ ]
```

Step 3 - Standardize HTTP status codes

The TaskController was updated to return standard HTTP status codes. GET requests return 200 OK, POST returns 201 Created, PUT returns 200 OK, and DELETE returns 204 No Content.

TaskController.php

```php
public function index(): JsonResponse
{
    $tasks = Cache::remember('tasks.index', 60, function () {
        return Task::all();
    });

    return response()->json($tasks, 200);
}

public function store(Request $request): JsonResponse
{
    $validated = $request->validate([
        'title' => 'required|string|max:255',
        'description' => 'nullable|string',
        'status' => 'nullable|string',
        'album_number' => 'required|string',
    ]);

    $task = Task::create($validated);
    Cache::forget('tasks.index');

    return response()->json($task, 201);
}
```

Step 4 – Introduce consistent error handling

I created a reusable API error helper to provide consistent JSON error responses. The helper returns the status code, error message, and request path in a structured format.

ApiError.php — backend > app > Support > ApiError.php

```php
<?php

namespace App\Support;

use Illuminate\Http\Request;
use Illuminate\Http\JsonResponse;

class ApiError
{
    public static function make(Request $request, int $status, string $message): JsonResp
    {
        return response()->json([
            'error' => [
                'status' => $status,
                'message' => $message,
                'path' => $request->getRequestUri(),
            ],
        ], $status);
    }
}
```

```
jad@fedora-2:~/uni-project/project5/backend$ php -l app/Support/ApiError.php
No syntax errors detected in app/Support/ApiError.php
```

Step 5 — Handle validation errors properly

Laravel validation was tested by sending an empty JSON body to the POST endpoint. Since required fields were missing, the API returned a 422 validation error with a structured JSON response.

```
jad@fedora-2:~/uni-project/project5$ curl -i -X POST
http://127.0.0.1:8080/api/78706/v1/tasks -H "Content-Type: application/json" -H "Accept:
application/json" -d '{}'
HTTP/1.1 422 Unprocessable Content
Server: nginx/1.27.5
Date: Sun, 30 Aug 2026 14:35:36 GMT
Content-Type: application/json

{"message":"The title field is required. (and 1 more error)","errors":{"title":["The title
field is required."],"album_number":["The album_number field is required."]}}
```

Step 6 — Handle missing resources

Missing resources were handled using Task::find($id). When a task does not exist, the API returns a 404 Not Found response with the standardized error format.

```
jad@fedora-2:~/uni-project/project5$ curl -i -H "Accept: application/json"
http://127.0.0.1:8080/api/78706/v1/tasks/99999
HTTP/1.1 404 Not Found
Server: nginx/1.27.5
Date: Sun, 30 Aug 2026 15:05:12 GMT
Content-Type: application/json

{"error":{"status":404,"message":"Task not found","path":"/api/78706/v1/tasks/99999"}}
```

Step 7 — Ensure cache still works after changes

Redis caching was tested after introducing API versioning and standardized responses. The task list endpoint repopulates the cache after a GET request, and write operations invalidate the cache.

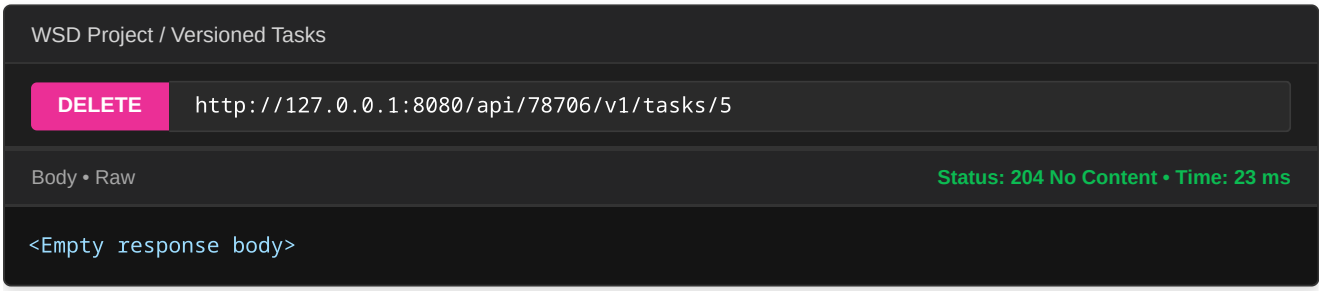
```
jad@fedora-2:~/uni-project/project5$ docker compose exec redis redis-cli -n 1 FLUSHDB
OK
jad@fedora-2:~/uni-project/project5$ docker compose exec redis redis-cli -n 1 DBSIZE
(integer) 0
jad@fedora-2:~/uni-project/project5$ curl http://127.0.0.1:8080/api/78706/v1/tasks
[]
jad@fedora-2:~/uni-project/project5$ docker compose exec redis redis-cli -n 1 DBSIZE
(integer) 1
```

Step 8 — Test full API flow

The full API flow was tested using GET, POST, PUT, and DELETE requests. All operations returned the expected status codes and JSON responses without crashes.

```
jad@fedora-2:~/uni-project/project5$ curl -i -X POST
http://127.0.0.1:8080/api/78706/v1/tasks -H "Content-Type: application/json" -d
'{"title":"Full flow test","description":"Step 8
test","status":"pending","album_number":"78706"}'
HTTP/1.1 201 Created
Content-Type: application/json

{"id":4,"title":"Full flow test","description":"Step 8
test","status":"pending","album_number":"78706","created_at":"2026-08-
30T16:39:14.000000Z","updated_at":"2026-08-30T16:39:14.000000Z"}

jad@fedora-2:~/uni-project/project5$ curl -i -X PUT
http://127.0.0.1:8080/api/78706/v1/tasks/4 -H "Content-Type: application/json" -d
'{"title":"Updated full flow test","status":"done"}'
HTTP/1.1 200 OK
Content-Type: application/json

{"id":4,"title":"Updated full flow test","description":"Step 8
test","status":"done","album_number":"78706","created_at":"2026-08-
30T16:39:14.000000Z","updated_at":"2026-08-30T16:39:20.000000Z"}

jad@fedora-2:~/uni-project/project5$ curl -i -X DELETE
http://127.0.0.1:8080/api/78706/v1/tasks/4
HTTP/1.1 204 No Content
```

Postman Testing - DELETE Request



Postman Testing - PUT Request

WSD Project / Versioned Tasks

PUT http://127.0.0.1:8080/api/78706/v1/tasks/5

```
{
  "title": "Updated Postman full flow test",
  "description": "Updated Step 8 Postman test",
  "status": "done",
  "album_number": "78706"
}
```

Body • Pretty • JSON — Status: 200 OK • Time: 29 ms


```
{
    "id": 5,
    "title": "Updated Postman full flow test",
    "description": "Updated Step 8 Postman test",
    "status": "done",
    "created_at": "2026-08-30T16:39:14.000000Z",
    "updated_at": "2026-08-30T16:42:48.000000Z"
}
```


Postman Testing - POST Request

WSD Project / Versioned Tasks

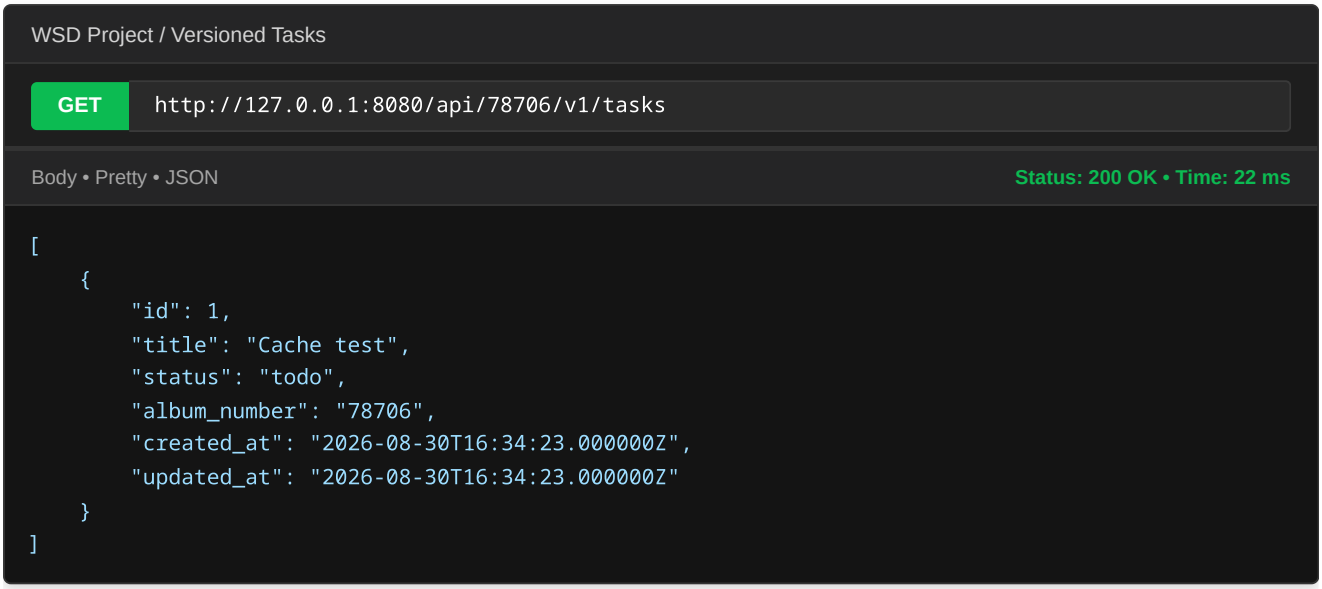
POST http://127.0.0.1:8080/api/78706/v1/tasks

```
{
  "title": "Postman full flow test",
  "description": "Step 8 Postman test",
  "status": "pending",
  "album_number": "78706"
}
```

Body • Pretty • JSON — Status: 201 Created • Time: 21 ms


```
{
    "id": 5,
    "title": "Postman full flow test",
    "description": "Step 8 Postman test",
    "status": "pending",
    "album_number": "78706",
    "updated_at": "2026-08-30T16:41:49.000000Z",
    "created_at": "2026-08-30T16:41:49.000000Z"
}
```

Postman Testing - GET Versioned Tasks List



Step 9 — Update documentation

The README file was updated to document the current API version, available endpoints, expected HTTP status codes, error responses, cache behavior, and full API flow test commands.

README.md

```
# Laboratory 5 API Documentation

## API Version
Current version: `/api/78706/v1`

Base endpoint:
http://127.0.0.1:8080/api/78706/v1/tasks

Health endpoint:
http://127.0.0.1:8080/api/health

## Status Codes
- `200 OK` - successful read or update operation
- `201 Created` - resource successfully created
- `204 No Content` - resource successfully deleted
- `404 Not Found` - requested resource does not exist
- `422 Unprocessable Content` - validation error
```

Step 10 — Commit changes

After completing the laboratory tasks, I committed, and pushed to the GitHub repository.

```
jad@fedora-2:~/uni-project/project5$ git status
On branch main
Changes to be committed:
  modified: routes/api.php
  modified: app/Http/Controllers/Api/TaskController.php
  new file: app/Support/ApiError.php

jad@fedora-2:~/uni-project/project5$ git commit -m "Project 5: API versioning and
standardization"
[main 9c2ecba] Project 5: API versioning and standardization
3 files changed, 68 insertions(+), 6 deletions(-)

jad@fedora-2:~/uni-project/project5$ git push
To https://github.com/amjadmassaoud/wsd-project.git
e6ae964..9c2ecba main -> main
```

API versioning

API versioning means placing API routes under a versioned URL prefix, such as **/api/78706/v1**. This makes the API easier to maintain because future changes can be introduced in a new version without breaking existing applications that still depend on the old version. In this project, the task endpoints were moved under /api/78706/v1/tasks.

HTTP status codes

HTTP status codes are used to clearly describe the result of each API request. In this laboratory, successful GET and PUT requests return **200 OK**, POST returns **201 Created**, DELETE returns **204 No Content**, missing resources return **404 Not Found**, and validation errors return **422 Unprocessable Content**. Using correct status codes makes the API easier to understand, debug, and integrate with frontend applications.

Validation errors

Validation errors occur when the client sends incomplete or invalid data. In this project, fields such as title and album_number are required when creating a task. If these fields are missing, Laravel returns **422 Unprocessable Content** together with a JSON response explaining which fields are invalid. This prevents invalid data from being saved to the database.

Why consistency in API design matters

Consistency in API design is important because it makes the system predictable and easier to use. When all endpoints follow the same URL structure, response format, error format, and status code rules, frontend developers can integrate with the API more easily. It also improves debugging, maintenance, and future development because every response follows the same pattern.